Guía de solución - Caso 3: Procesador de pagos multicanal

Elaborado por: Henry Antonio Mendoza Puerta

Enunciado

El caso solicita un procesador de pagos con varios métodos (tarjeta, PayPal, criptomonedas y billeteras móviles), aplicando el patrón Strategy:

- Interfaz PaymentStrategy con el método void pay(double amount).

- Estrategias concretas: CreditCardPaymentStrategy, PayPalPaymentStrategy, CryptoPaymentStrategy y MobileWalletPaymentStrategy.

- Contexto (PaymentProcessor) que delega el pago a la estrategia seleccionada.

Cómo aplicamos el patrón Strategy

Cada medio de pago (tarjeta, PayPal, criptomonedas y billetera móvil) es una clase que implementa la interfaz Pago.

El contexto ProcesadorPago recibe una estrategia y realiza el pago sin saber con qué medio.

ServicioPago elige la estrategia según el método seleccionado.

Si se agrega un nuevo medio de pago, solo se crea una clase nueva y se registra en el servicio: el procesador no cambia.

En esta solución: PaymentStrategy = Pago, PaymentProcessor = ProcesadorPago, pay = pagar (devuelve el mensaje de confirmación para mostrarlo).

Paquetes del proyecto

Paquete	Contenido
model	Clases de datos y enumerados: MetodoPago.
strategy	El patrón: interfaz Pago, los cuatro medios de pago y el contexto ProcesadorPago.
service	Elige la estrategia según el método de pago y paga (ServicioPago).
view	La ventana (JFrame) en VistaPago.
controller	ControladorPago: conecta la ventana con el servicio.
(raíz)	Principal: inicia el programa.

Paso a paso

Paso 1. Modelo (MetodoPago)

Enumerado con los métodos de pago.

Archivo: MetodoPago.java

```
package com.usil.model;

public enum MetodoPago {
    TARJETA, PAYPAL, CRIPTO, BILLETERA
}
```

Paso 2. Interfaz de la estrategia (Pago)

Recibe el monto y devuelve el mensaje de confirmación.

Archivo: Pago.java

```
package com.usil.strategy;

public interface Pago {

    String pagar(double monto);
}
```

Paso 3. Estrategias concretas

Una clase por cada medio de pago.

Archivo: PagoTarjeta.java

```
package com.usil.strategy;

public class PagoTarjeta implements Pago {

    @Override
    public String pagar(double monto) {
        return "Pago de S/ " + monto + " realizado con tarjeta de crédito.";
    }
}
```

Archivo: PagoPayPal.java

```
package com.usil.strategy;

public class PagoPayPal implements Pago {

    @Override
    public String pagar(double monto) {
        return "Pago de S/ " + monto + " realizado con PayPal.";
    }
}
```

Archivo: PagoCripto.java

```
package com.usil.strategy;

public class PagoCripto implements Pago {

    @Override
    public String pagar(double monto) {
        return "Pago de S/ " + monto + " realizado con criptomonedas.";
    }
}
```

Archivo: PagoBilletera.java

```
package com.usil.strategy;

public class PagoBilletera implements Pago {

    @Override
    public String pagar(double monto) {
        return "Pago de S/ " + monto + " realizado con billetera móvil.";
    }
}
```

Paso 4. Contexto (ProcesadorPago)

Delega el pago en la estrategia elegida.

Archivo: ProcesadorPago.java

```
package com.usil.strategy;

/** Contexto: delega el pago en la estrategia seleccionada. */
public class ProcesadorPago {

    private Pago estrategia;

    public void setEstrategia(Pago estrategia) {
        this.estrategia = estrategia;
    }

    public String procesar(double monto) {
        return estrategia.pagar(monto);
    }
}
```

Paso 5. Servicio (ServicioPago)

Elige la estrategia según el método de pago.

Archivo: ServicioPago.java

```
package com.usil.service;

import com.usil.model.MetodoPago;
import com.usil.strategy.Pago;
import com.usil.strategy.PagoBilletera;
import com.usil.strategy.PagoCripto;
import com.usil.strategy.PagoPayPal;
import com.usil.strategy.PagoTarjeta;
import com.usil.strategy.ProcesadorPago;

public class ServicioPago {

    private ProcesadorPago procesador = new ProcesadorPago();

    public String pagar(MetodoPago metodo, double monto) {
        Pago estrategia = switch (metodo) {
            case TARJETA -> new PagoTarjeta();
            case PAYPAL -> new PagoPayPal();
            case CRIPTO -> new PagoCripto();
            case BILLETERA -> new PagoBilletera();
        };
        procesador.setEstrategia(estrategia);
        return procesador.procesar(monto);
    }
}
```

Paso 6. Crear la vista (VistaPago)

Clic derecho sobre view, New, JFrame Form, nombre VistaPago. En Diseño: JFrame con Border Layout, un JPanel arriba (North) y un JLabel al centro (Center). Agregar al panel estos componentes y renombrarlos (clic derecho, Change Variable Name):

Nombre	Componente	Texto
cmbMetodo	JComboBox	(vacío)
txtMonto	JTextField	(vacío)
btnPagar	JButton	Pagar
lblResultado	JLabel	(vacío)

En Código, fuera del bloque Generated Code, agregar mostrarError y los getters. Archivo completo:

Archivo: VistaPago.java

```
package com.usil.view;

import javax.swing.JOptionPane;

public class VistaPago extends javax.swing.JFrame {

    public VistaPago() {
        initComponents();
        setSize(950, 160);
    }

    public void mostrarError(String message) {
        JOptionPane.showMessageDialog(this, message, "Datos no válidos",
JOptionPane.WARNING_MESSAGE);
    }

    public javax.swing.JComboBox<String> getCmbMetodo() {
        return cmbMetodo;
    }

    public javax.swing.JTextField getTxtMonto() {
        return txtMonto;
    }

    public javax.swing.JButton getBtnPagar() {
        return btnPagar;
    }

    public javax.swing.JLabel getLblResultado() {
        return lblResultado;
    }

    /** Inicializa los componentes del formulario. No modificar: lo regenera el editor de
NetBeans (pestaña Diseño). */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated
Code">//GEN-BEGIN:initComponents
    private void initComponents() {

        pnlFormulario = new javax.swing.JPanel();
        lblMetodo = new javax.swing.JLabel();
        cmbMetodo = new javax.swing.JComboBox<>();
        lblMonto = new javax.swing.JLabel();
        txtMonto = new javax.swing.JTextField();
        btnPagar = new javax.swing.JButton();
        lblResultado = new javax.swing.JLabel();

        setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);
        setTitle("Caso 3 - Procesador de pagos");

        lblMetodo.setText("Método de pago:");
        pnlFormulario.add(lblMetodo);

        cmbMetodo.setModel(new javax.swing.DefaultComboBoxModel<>(new String[] { }));
        pnlFormulario.add(cmbMetodo);

        lblMonto.setText("Monto:");
        pnlFormulario.add(lblMonto);

        txtMonto.setColumns(8);
        pnlFormulario.add(txtMonto);

        btnPagar.setText("Pagar");
        pnlFormulario.add(btnPagar);
```

```
        lblResultado.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
        lblResultado.setText(" ");

        getContentPane().add(pnlFormulario, java.awt.BorderLayout.NORTH);
        getContentPane().add(lblResultado, java.awt.BorderLayout.CENTER);

        pack();
    }// </editor-fold>//GEN-END:initComponents

    // Variables declaration - do not modify//GEN-BEGIN:variables
    private javax.swing.JButton btnPagar;
    private javax.swing.JComboBox<String> cmbMetodo;
    private javax.swing.JLabel lblMetodo;
    private javax.swing.JLabel lblMonto;
    private javax.swing.JLabel lblResultado;
    private javax.swing.JPanel pnlFormulario;
    private javax.swing.JTextField txtMonto;
    // End of variables declaration//GEN-END:variables
}
```

Paso 7. Controlador (ControladorPago)

Llena el combo, valida el monto, escucha el botón y muestra la confirmación.

Archivo: ControladorPago.java

```
package com.usil.controller;

import com.usil.model.MetodoPago;
import com.usil.service.ServicioPago;
import com.usil.view.VistaPago;
import java.util.Arrays;

public class ControladorPago {

    private VistaPago vista;
    private ServicioPago servicio;

    public ControladorPago(VistaPago vista, ServicioPago servicio) {
        this.vista = vista;
        this.servicio = servicio;

        // El combo se llena con los valores del enum usando streams.
        Arrays.stream(MetodoPago.values()).forEach(e ->
vista.getCmbMetodo().addItem(e.name()));

        vista.getBtnPagar().addActionListener(e -> pagar());
    }

    public void iniciar() {
        vista.setLocationRelativeTo(null);
        vista.setVisible(true);
    }

    private void pagar() {
        try {
            MetodoPago metodo =
MetodoPago.values()[vista.getCmbMetodo().getSelectedIndex()];
            double monto = Double.parseDouble(vista.getTxtMonto().getText().trim());
            if (monto <= 0) {
                vista.mostrarError("El monto debe ser mayor que 0.");
                return;
            }
            vista.getLblResultado().setText(servicio.pagar(metodo, monto));
        } catch (NumberFormatException ex) {
            vista.mostrarError("Ingrese un monto válido.");
        }
```

```
    }
}
```

Paso 8. Crear la clase Principal

Inicia el programa.

Archivo: Principal.java

```
package com.usil;

import com.usil.controller.ControladorPago;
import com.usil.service.ServicioPago;
import com.usil.view.VistaPago;
import javax.swing.SwingUtilities;
import javax.swing.UIManager;

public class Principal {

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            try {
                UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
            } catch (Exception ex) {
                // se usa el aspecto por defecto de Swing
            }
            new ControladorPago(new VistaPago(), new ServicioPago()).iniciar();
        });
    }
}
```

Paso 9. Ejecutar y probar

Clic derecho sobre Principal.java, Run File. Se ingresan los datos de prueba y se presiona el botón.

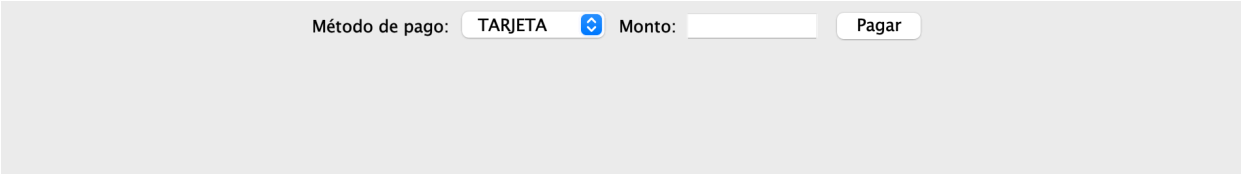


Figura 1. Ventana inicial.

Método	Monto	Resultado
TARJETA	150.75	Pago de S/ 150.75 realizado con tarjeta de crédito.
PAYPAL	150.75	Pago de S/ 150.75 realizado con PayPal.
CRIPTO	150.75	Pago de S/ 150.75 realizado con criptomonedas.
BILLETERA	150.75	Pago de S/ 150.75 realizado con billetera móvil.

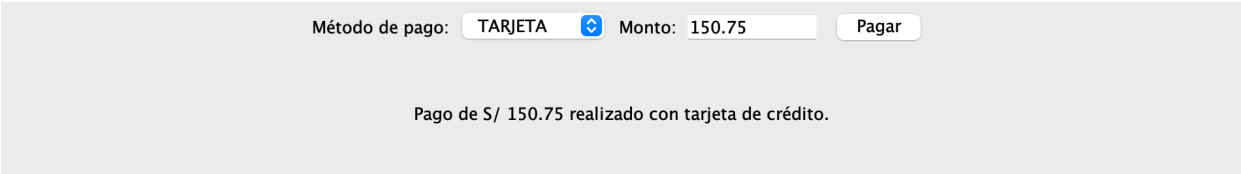


Figura 2. TARJETA.

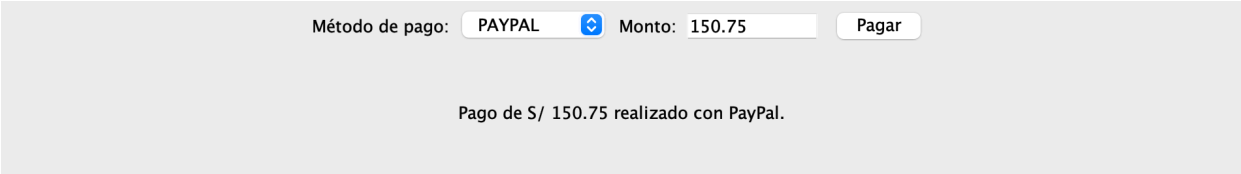


Figura 3. PAYPAL.

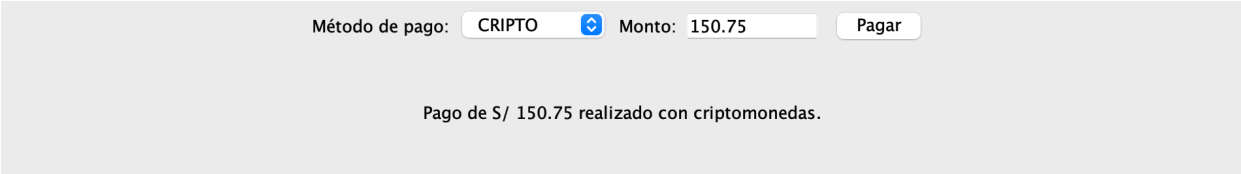


Figura 4. CRIPTO.

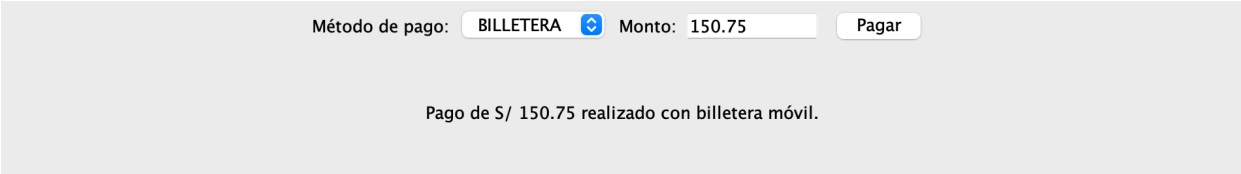


Figura 5. BILLETERA.